Set 2 — Assignment

1. Structured Data — CSV

Create:

`renewable_energy.csv`

Paste:

```
Plant_ID,Plant_Name,State,Energy_Type,Capacity_MW,Actual_Output_MW,Operat
P101,SunGrid One,Rajasthan,Solar,120,98,9,45000
P102,WindArc West,Gujarat,Wind,150,132,18,62000
P103,HydroPeak,Kerala,Hydro,200,176,20,78000
P104,SolarEdge,Telangana,Solar,100,81,8,38000
P105,WindCore,Tamil Nadu,Wind,180,149,17,71000
P106,RiverForce,Uttarakhand,Hydro,220,194,21,84000
P107,SunValley,Madhya Pradesh,Solar,140,112,10,52000
P108,CoastalWind,Odisha,Wind,160,138,19,66000
```

Create:

`structured_set2.py`

Assignments

1. Load the CSV into a Pandas DataFrame.
2. Display plants where `Energy_Type` is either `Solar` or `Wind`.
3. Create a new column:

```
Efficiency_Percent =
Actual_Output_MW / Capacity_MW * 100
```

4. Display only plants with efficiency above 85%.

5. Find the average efficiency for each energy type.
6. Find the total maintenance cost for each energy type.
7. Display the plant with the highest `Actual_Output_MW`.
8. Create a new column:

```
Cost_Per_MW =
Maintenance_Cost / Actual_Output_MW
```

9. Sort plants by `Cost_Per_MW` from lowest to highest.
10. Display only those plants where:

- efficiency is above 80%
- maintenance cost is below 70000

Extra challenge

Create a bar chart showing:

```
Plant_Name vs Efficiency_Percent
```

---

2. Semi-Structured Data – JSON

Create:

```
smart_home_devices.json
```

Paste:

```
[
  {
    "device_id": "D301",
    "device_type": "Thermostat",
    "room": "Living Room",
    "status": "Active",
    "metrics": {
      "temperature": 24,
```

```
      "battery": 82
    },
    "alerts": ["High usage"]
  },
  {
    "device_id": "D302",
    "device_type": "Camera",
    "room": "Entrance",
    "status": "Active",
    "metrics": {
      "battery": 65
    }
  },
  {
    "device_id": "D303",
    "device_type": "Air Quality Sensor",
    "room": "Bedroom",
    "status": "Inactive",
    "metrics": {
      "air_quality_index": 78,
      "battery": 40
    },
    "alerts": ["Battery low"]
  },
  {
    "device_id": "D304",
    "device_type": "Thermostat",
    "room": "Study",
    "status": "Active",
    "metrics": {
      "temperature": 27,
      "battery": 74
    }
  },
  {
    "device_id": "D305",
    "device_type": "Smart Lock",
    "room": "Back Door",
    "status": "Active",
    "metrics": {
      "battery": 55
    },
    "alerts": ["Firmware update available"]
```

```
    }
]
```

Create:

```
semi_structured_set2.py
```

Assignments

1. Load the JSON file.
2. Print:
    - device_id
    - device_type
    - room
3. Display only devices whose `status` is `Active`.
4. Display devices whose battery level is below 60.
5. Print the temperature only for devices that actually contain a `temperature` field.
6. Display devices that contain an `alerts` field.
7. Count how many devices exist for each `device_type`.
8. Add a new field:

```
health
```

Rule:

```
battery >= 70 → Good
battery 50-69 → Moderate
battery < 50 → Low
```

9. Save the updated JSON into:

```
device_health_report.json
```

10. Convert the JSON into a DataFrame using `pd.json_normalize()` and display:

```
device_id
device_type
room
status
metrics.battery
health
```

Extra challenge

Display only:

```
Active devices with battery below 70
```

---

3. Unstructured Data — Text

Create:

```
airport_operations_report.txt
```

Paste:

```
Flight operations were delayed during the early morning because of dense

Ground staff reported reduced visibility and several aircraft were kept a

The weather improved gradually after 9 AM and departures resumed in phase

One aircraft required an additional technical inspection before departure

Passengers on delayed flights were informed through the airport notificat

The northern runway remained fully operational throughout the incident.

No emergency situations were reported.

Airport management requested a detailed review of the delays and communic

A follow-up meeting has been scheduled with the operations, weather, and
```

Create:

```
unstructured_set2.py
```

Assignments

1. Read the complete report.
2. Count:
    - total words
    - total lines
    - total characters
3. Count how many times the words below occur:

```
flight
runway
operations
reported
```

4. Print only lines containing the word:

```
runway
```

5. Print only lines containing either:

```
delay
delayed
delays
```

6. Extract all lines that mention something being `reported`.
7. Check whether these words appear:

```
emergency
inspection
weather
passengers
maintenance
```

8. Create a dictionary like:

```
{
    "flight": count,
    "runway": count,
    "operations": count
}
```

9. Find the longest line in the report.

10. Save only the lines related to delays or weather into:

```
filtered_airport_report.txt
```